Chain of Responsibility

Season: 3 · Behavioral · Category: GoF · Level: Intermediate · Build with Waqar

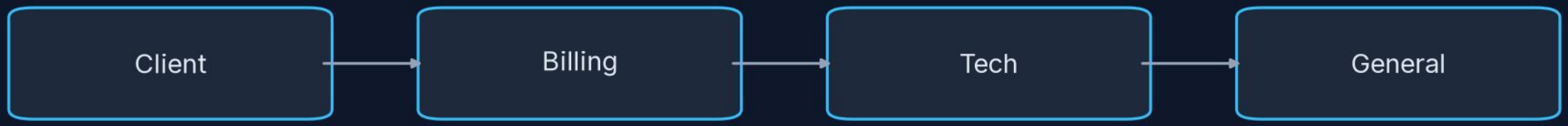
Today's Hook

- Today: Chain of Responsibility as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Chain of Responsibility structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Chain of Responsibility
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/behavioral/chain/SupportHandler.java
package com.buildwithwaqar.patterns.gof.behavioral.chain;

public abstract class SupportHandler {
    private SupportHandler next;

    public SupportHandler linkWith(SupportHandler next) {
        this.next = next;
        return next;
    }

    public final String handle(String request) {
        var result = doHandle(request);
        if (result != null) return result;
        return next != null ? next.handle(request) : "unhandled:" + request;
    }

    protected abstract String doHandle(String request);
}

final class BillingHandler extends SupportHandler {
    protected String doHandle(String request) {
        return request.startsWith("bill:") ? "billing-ok" : null;
    }
```

Implementation (2)

```
}

final class TechHandler extends SupportHandler {
    protected String doHandle(String request) {
        return request.startsWith("tech:") ? "tech-ok" : null;
    }
}

final class GeneralHandler extends SupportHandler {
    protected String doHandle(String request) { return "general-ok"; }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.gof.creational.builder;

public final class Demo {
    public static void main(String[] args) {
        System.out.println("=== Builder walkthrough ===");
        var req = HttpRequest.builder("https://api.buildwithwaqar.dev/v1/videos")
                .method("POST")
                .header("Authorization", "Bearer demo")
                .header("Content-Type", "application/json")
                .body("{\"title\":\"Singleton\"}")
                .timeoutMs(2500)
                .build();
        System.out.println("method=" + req.method());
        System.out.println("url=" + req.url());
        System.out.println("auth=" + req.headers().get("Authorization"));
        System.out.println("timeout=" + req.timeoutMs());
        var call = ApiCall.get("https://x").withBody("hi").withTimeout(100);
        System.out.println("apiCall=" + call.method() + " body=" + call.body() + " t=" + call.timeoutMs());
    }
}
```

Walkthrough Output

```
=== Chain walkthrough ===
bill:42 -> billing-ok
tech:wifi -> tech-ok
hello -> general-ok
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Chain of Responsibility
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Chain of Responsibility → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.